the clear choice
WWW.EDTC.COM

Smart Embedded ®

# Smart Embedded Evaluation Kit Getting Started Guide

# EVK043027B
# EVK050027B
# EVK057027B
# EVK070027B
# V1.0

Preliminary

Written by: Channing Hu

Reviewed by: Paul Kleist

Revision history:

| Date | Page, change | Version |
|---|---|---|
| 2026.02.4 | | Vprel |
| | | |
| | | |

Preliminary

## Legal Disclaimer:

THIS INFORMATION / SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

# Introduction

**The Smart Embedded Evaluation Kit is intended to give the user a Jump Start in developing Applications for the Smart Embedded Modules.**

**The Evaluation Kit contains all necessary Hardware, Project Example and Documentation to develop applications, simulate them and download them on the Evaluation Module.**

**The Kit comes Pre-programmed with a Demo Application. The Demo Application will run just by applying Power to the Module.**

**This Guide describes the Demo Application in detail and describes how the User can modify parameters for the On-Board Interfaces.**

Preliminary

# Table of Content


Legal Disclaimer: ..... 2
Table of Content ..... 4
1. Evaluation Kit Description ..... 5
1.1 General ..... 5
1.2 Picture of Evaluation Kit ..... 5
1.3 Example picture of PCB's ..... 6
2. Installation of tools and software ..... 7
2.1 Download ..... 7
2.1.1 TouchGFX, STM32CubeIDE and STM32CubeMX ..... 7
2.2 How to use TouchGFX ..... 8
2.2.1 Official website TouchGFX ..... 8
2.2.2 Documentation for all UI components of TouchGFX ..... 8
3. How to flash firmware to the EDT Smart Embedded Modules ..... 9
3.1 TouchGFX designer ..... 9
3.2 STM32CubeIDE ..... 9
3.2.1 Download EDT flash loader ..... 9
3.2.2 Debug mode of STM32CubeIDE ..... 10
3.3 Flashing firmware with STM32CubeProgrammer outside of IDE ..... 11
3.3.1 Download EDT flash loader ..... 11
3.3.2 ExternalLoader folder of STM32CubeProgrammer ..... 11
3.3.3 Flashing firmware ..... 12
4. Directory Structure of the Project ..... 13
4.1 Detailed Program Flow ..... 14
4.2 Enabling On-board Interfaces ..... 15
4.3 Low level Drivers for the On-board Interfaces ..... 16
5. The TouchGFX Project ..... 17
5.1 Main TouchGFX Designer Screen ..... 18
5.2 Run the Project on the Target Hardware ..... 19
5.3 Connection Diagram for RS485 and CAN Interface ..... 20
5.4 RS485 Screen ..... 21
5.5 CAN Screen ..... 24
5.6 Flash Test Screen ..... 26
5.7 SD Card Test ..... 28
5.8 USB CDC Screen ..... 31
5.9 Information Screen ..... 32
6. Expansion connector ..... 33


Preliminary

# 1.Evaluation Kit Description

## 1.1 General

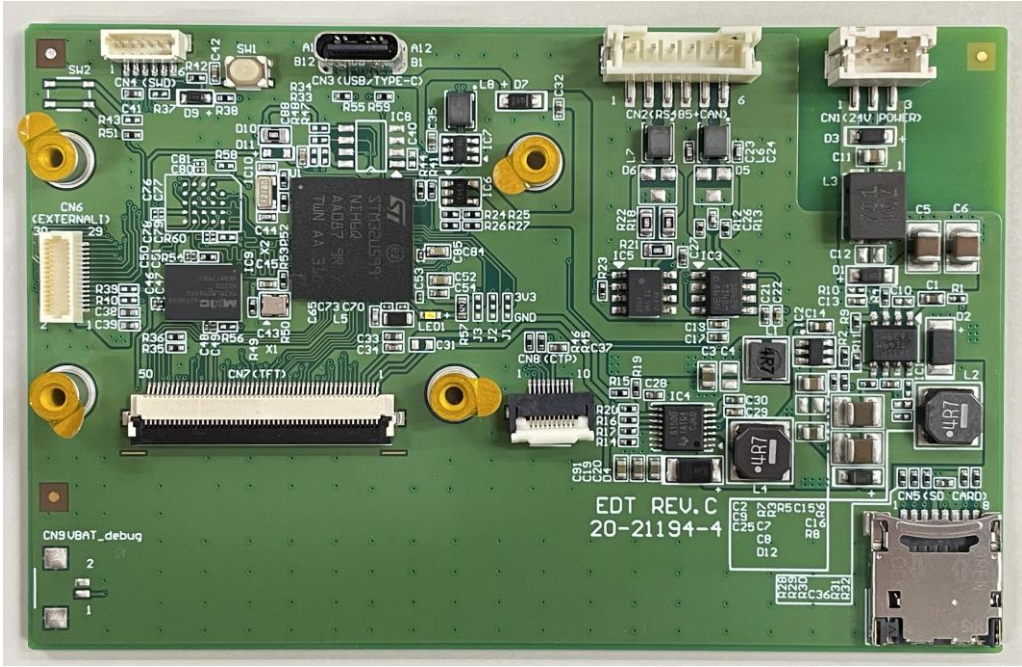
The Evaluation Kit consists of:

- Smart Embedded Module
- 2 set Expansion boards
  (1) Fanout Board (passive, for easy access to pins, solder wiring / 0.1“ connector)
  (2) 27 Series connector board with 5 pcs. 6-pin connectors with RS323 UART levels and 3.3V UARTx2, I2C, SPI and 4-bit GPIO.
- Power Supply Cable
- 7 extra interface cables
  3 pcs for SE Board:
  3 pin cable for power supply wire
  6 pin cable for interface (1pin head)
  6 pin cable for interface (2pin head)
  4 pcs 6pin interface cable for expansion board
- ST-Link V2 USB Programmer with Cable to on-board ST-LINK connector
- USB Cable for ST-LINK Programmer

## 1.2 Picture of Evaluation Kit

Preliminary

## 1.3 Example picture of PCB's




Preliminary

# 2. Installation of tools and software

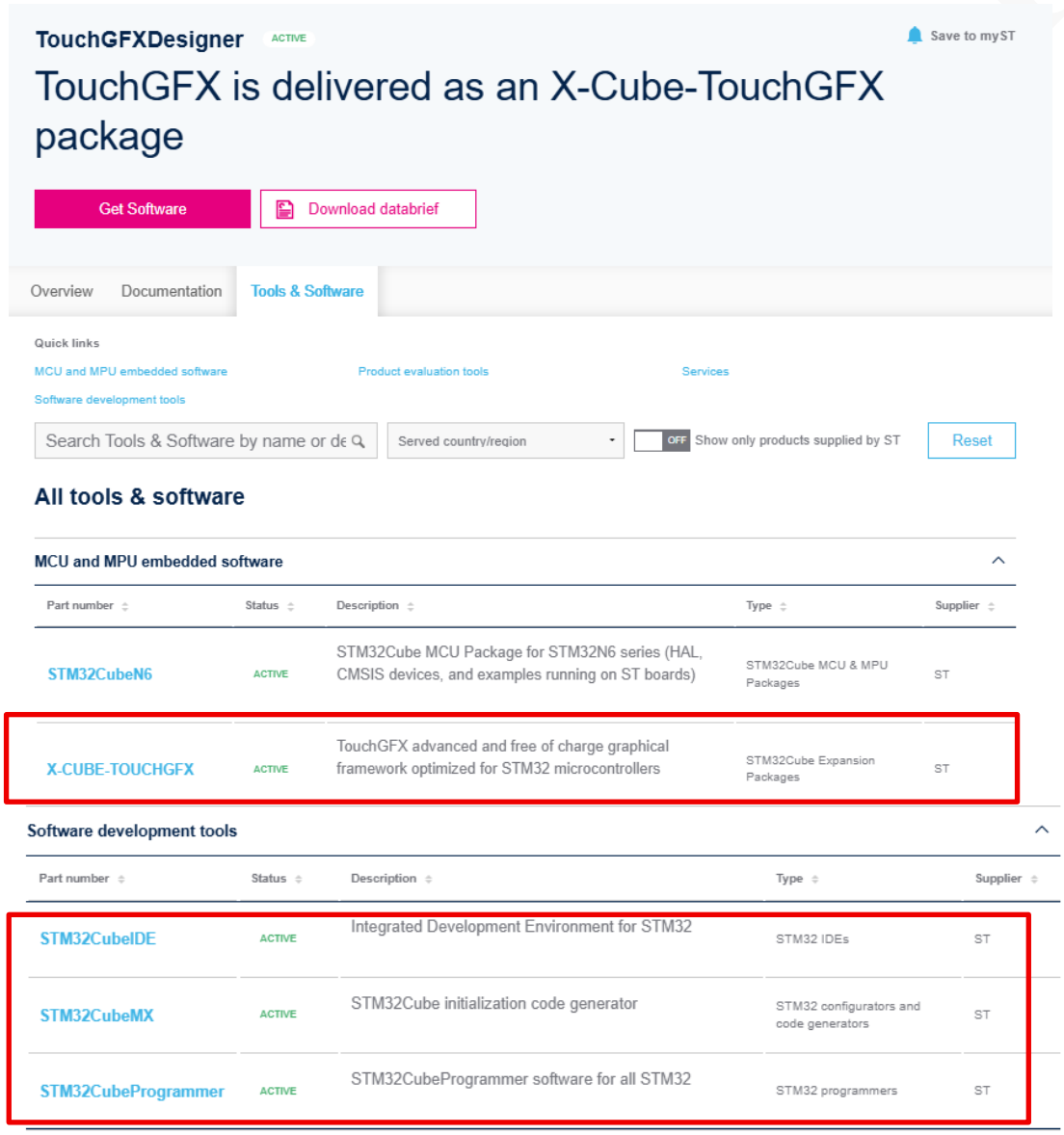
You can install free evaluation versions of STMicroelectronics tools and software to use in evaluating the performance of the EDT Smart Embedded Module with TouchGFX.

## *2.1 Download*

### 2.1.1 TouchGFX, STM32CubeIDE and STM32CubeMX

Go to https://www.st.com/en/development-tools/touchgfxdesigner.html#tools-software

Select 'Tools & Software'



## 2.2 How to use TouchGFX

### 2.2.1 Official website TouchGFX

How to build TouchGFX project step by step:
Graphics on STM32 Microcontrollers - STMicroelectronics
TouchGFX Documentation | TouchGFX Documentation

TouchGFX YouTube - UI Development on STM32:
TouchGFX Designer: Design, Implement, Run!

Watch some instruction videos to get an idea of how Designer works. It is highly recommended to develop a few Projects using Buttons to navigate to different Screens, using sliders and other widgets to get familiar with Designer and Interactions.

### 2.2.2 Documentation for all UI components of TouchGFX

This section contains information on every UI component found in TouchGFX - from widgets to containers:
UI Components | TouchGFX Documentation

Once this is done, we are ready to work with the Evaluation Kit example.

Preliminary

# 3. How to flash firmware to the EDT Smart Embedded Modules

There are multiple ways to flash firmware to EDT Smart Embedded Modules after programming and compiling.

## *3.1 TouchGFX designer*

Click the button “Run Target” in the lower right corner.

## *3.2 STM32CubeIDE*

### 3.2.1 Download EDT flash loader

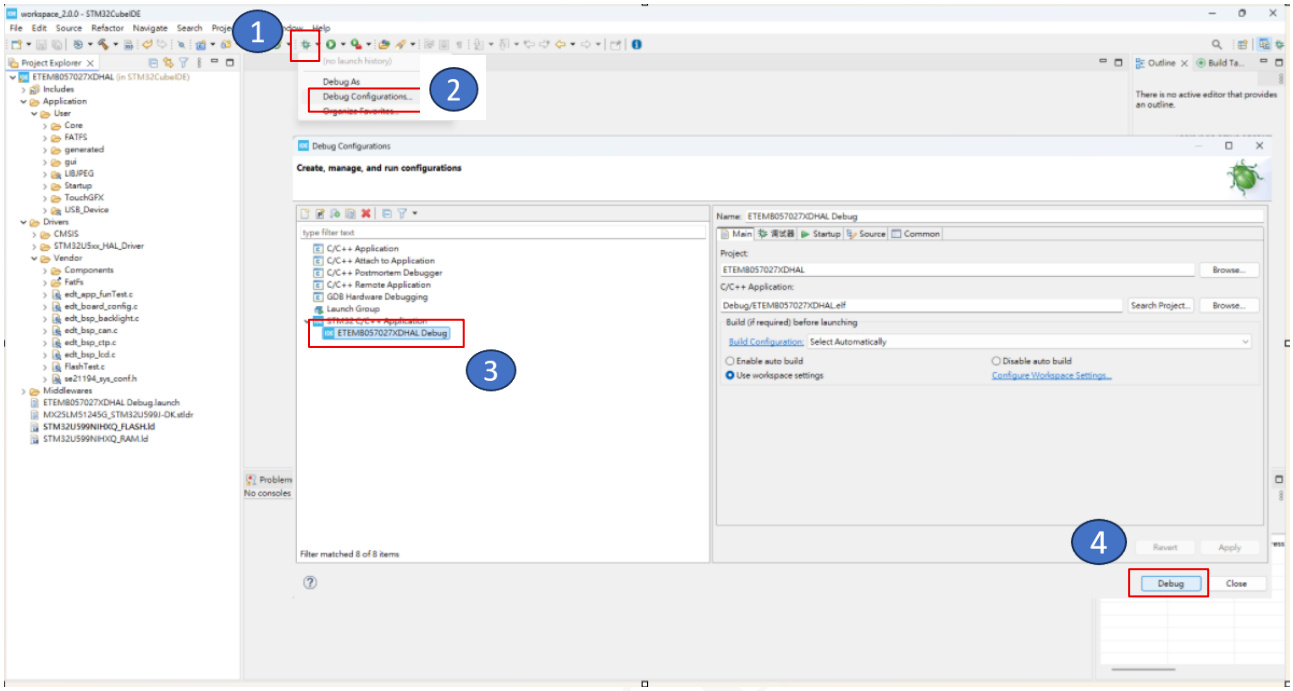
Copy the stldr file from path: \EVKxxx \Drivers\vendor\Board\SE xxx \Flashloader.

| Eval Kit part number | MCU part number | Flash loader part number |
|---|---|---|
| EVK043027B | U599 | MX25LM51245G_STM32U599J-DK.stldr |
| EVK050027B | | |
| EVK057027B | | |
| EVK070027B | | |

To: C:\Program Files\STMicroelectronics\STM32Cube\STM32CubeProgrammer\bin\ExternalLoader

### 3.2.2 Debug mode of STM32CubeIDE

Please refer to the steps below.




Prelim

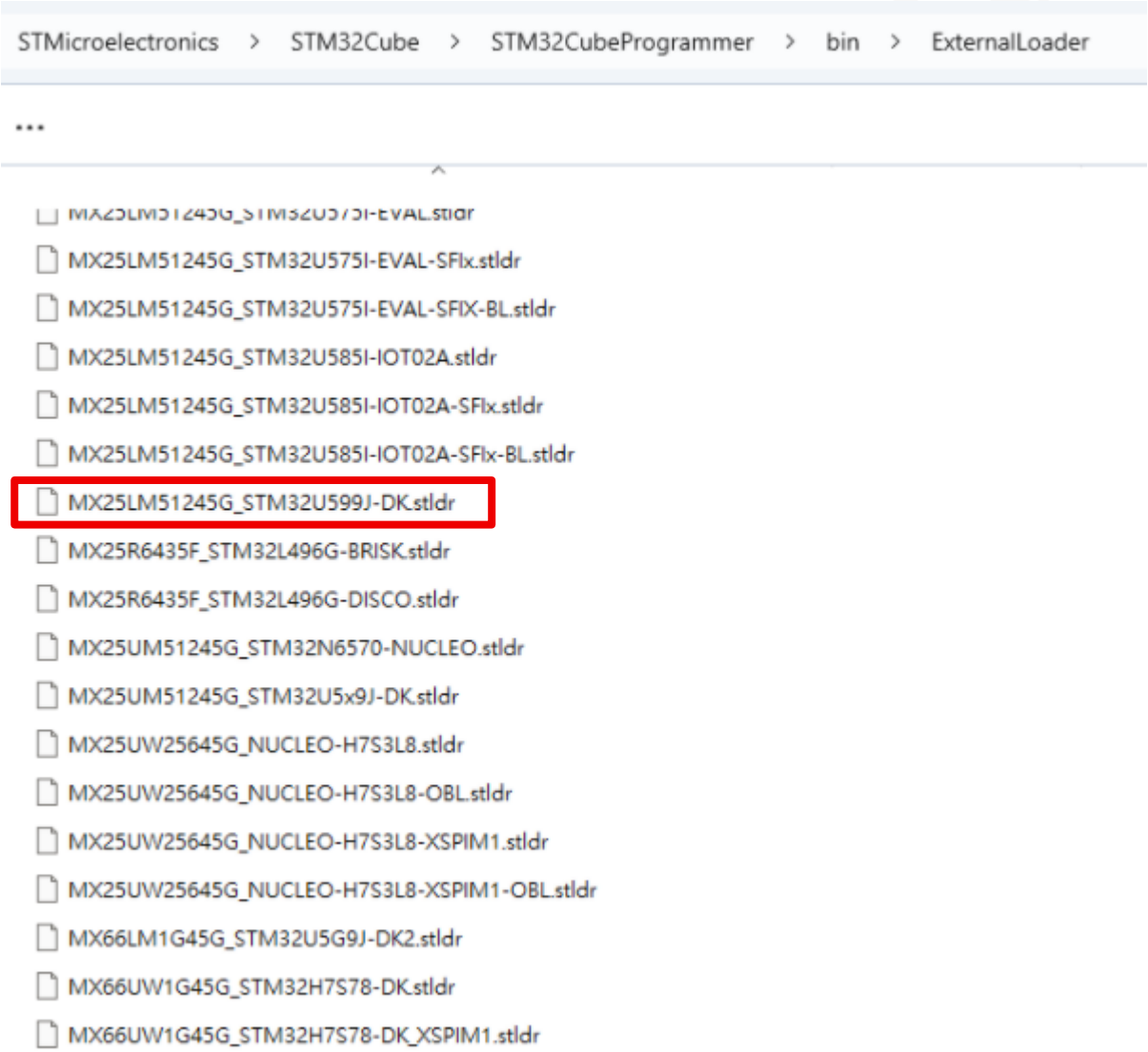
## 3.3 Flashing firmware with STM32CubeProgrammer outside of IDE

### 3.3.1 Download EDT flash loader

Refer to 3.2.1

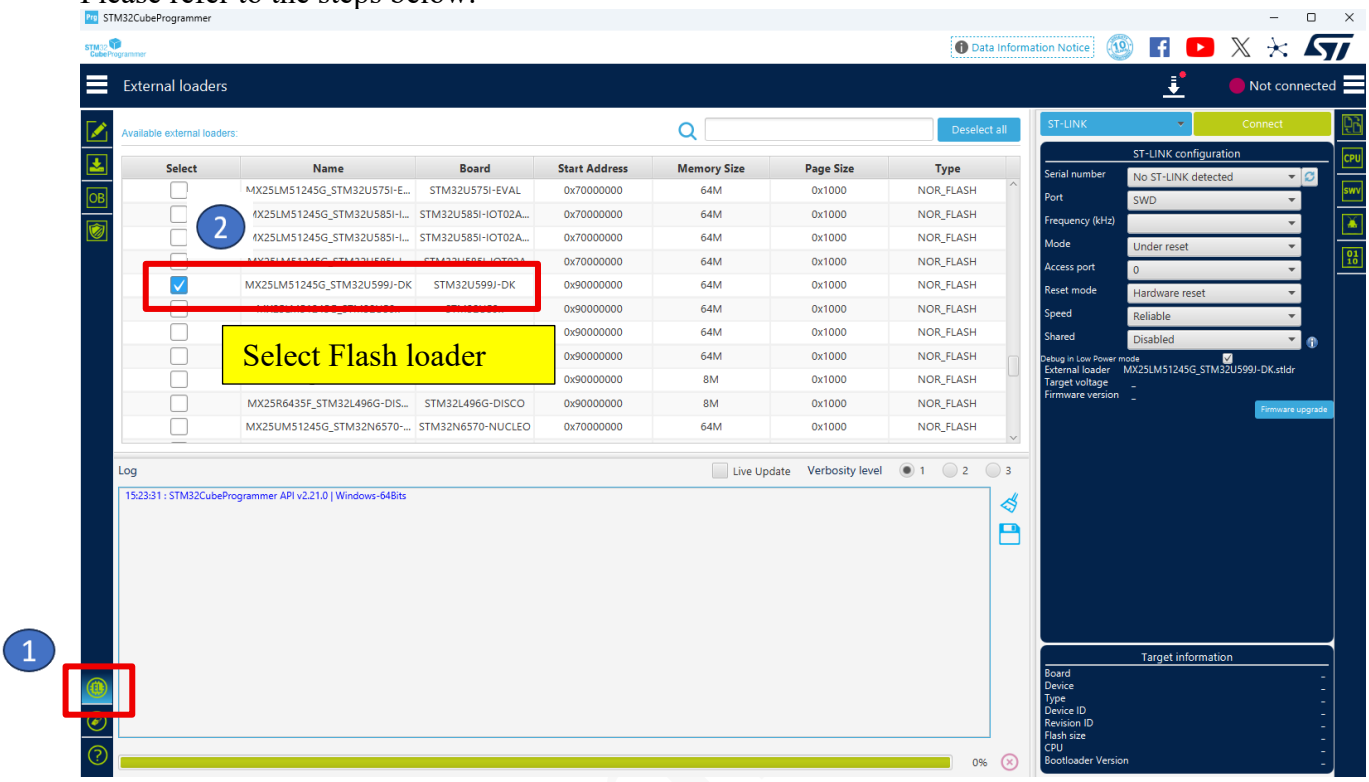
### 3.3.2 ExternalLoader folder of STM32CubeProgrammer

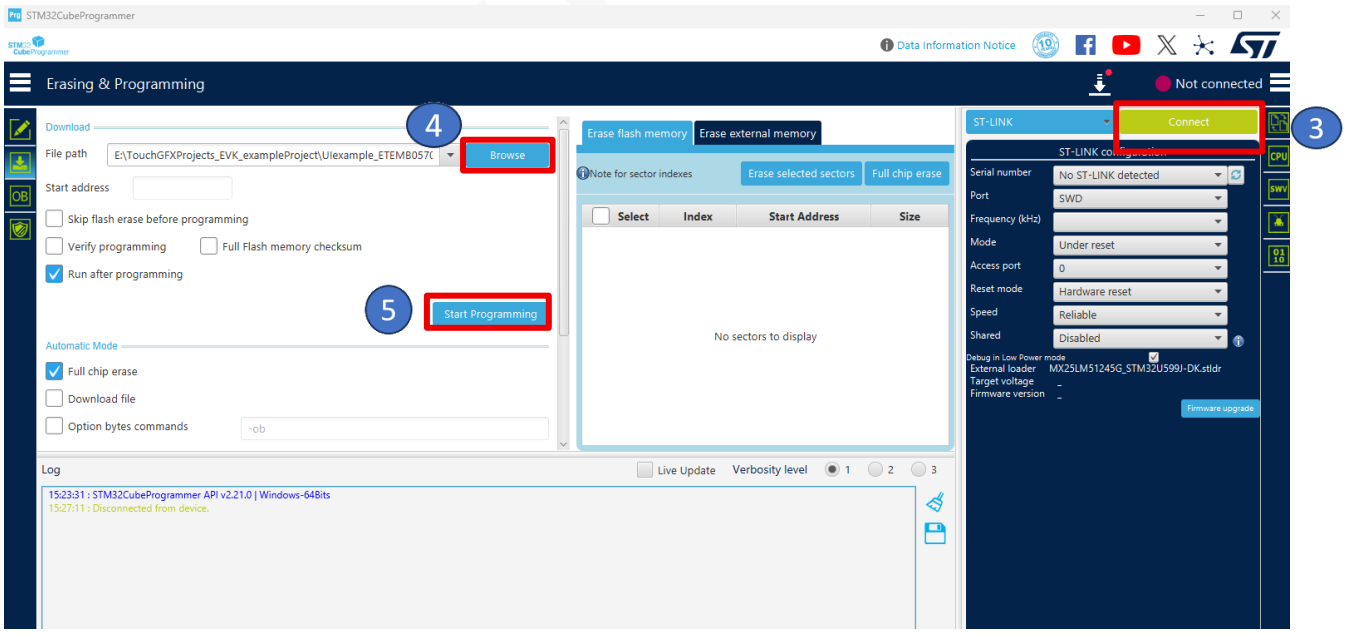
Copy the stldr file to
path: ..\STMicroelectronics\STM32Cube\STM32CubeProgrammer\bin\ExternalLoader.



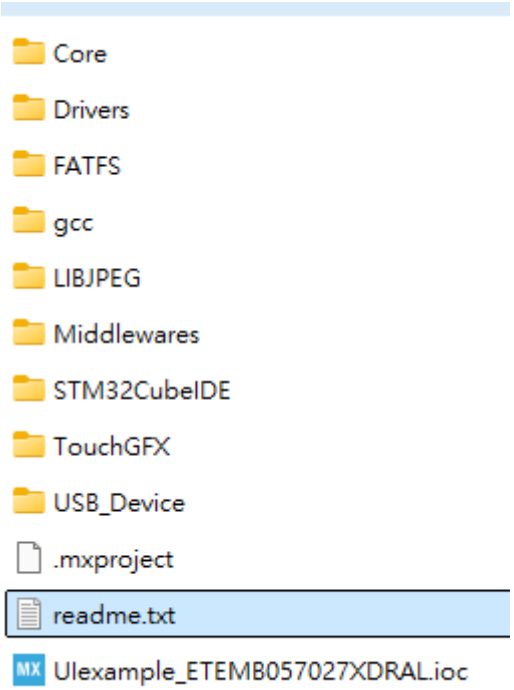
### 3.3.3 Flashing firmware

Please refer to the steps below.



# 4. Directory Structure of the Project



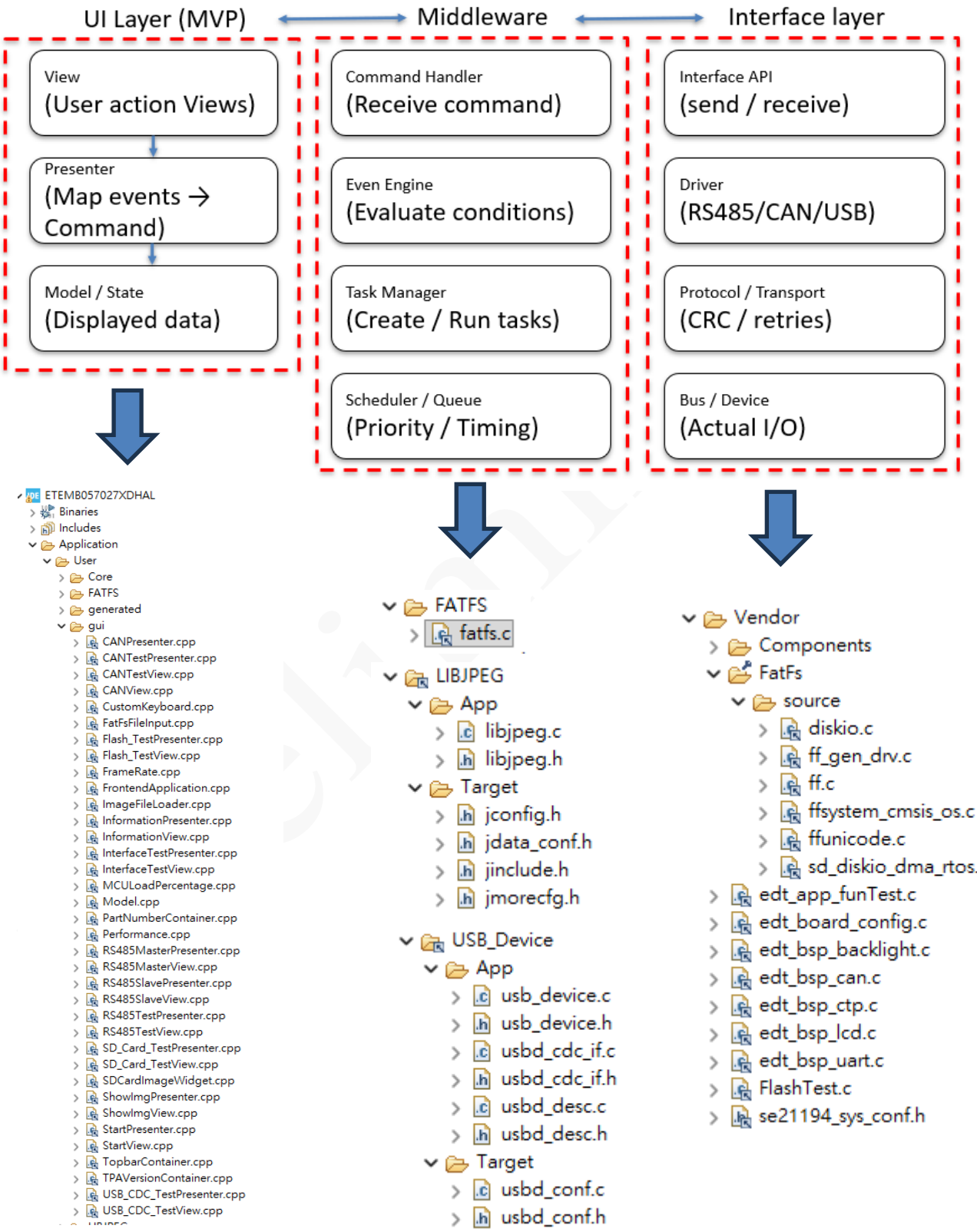

Directories of interest for the Project are:

- **Core** where you will find **main.c** and other files that are generated by CubeMX.
- **Drivers\Vendor\Board\, Drivers\Vendor\Device**, **Drivers\Vendor\Driver, FATFS** and **LIBJPEG** where you will find low level drivers from EDT.
- **gcc** contains the gcc **Makefile** and Flash loader.
- **TouchGFX** contains the **TouchGFX project.**
- **USB_DEVICE** contains USB related files (CDC class is implemented).

**TouchGFX is owned by STMicroelectronics. For information visit: www.st.com/stm32gui or www.touchgfx.com*
*TouchGFX is a registered and/or unregistered trademark of STMicroelectronics International NV or its affiliates in the EU and/or elsewhere. In particular, TouchGFX is Registered in the US Patent and Trademark Office*

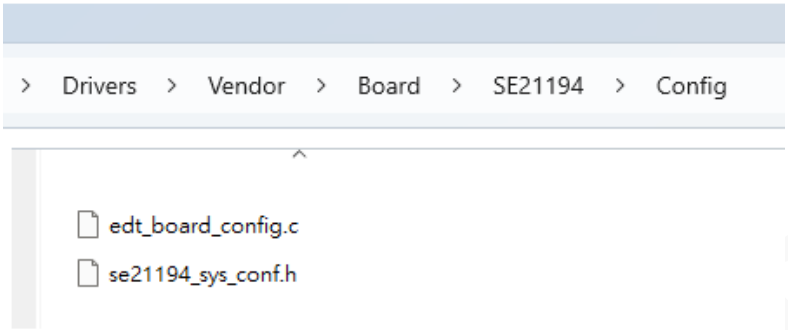
Preliminary

## 4.1 Detailed Program Flow



## 4.2 Enabling On-board Interfaces

The Interfaces can be enabled/disabled in the ***se* yyyy*_sys_conf.h*** file in the following directory:
path: Project xxx/Drivers/Vendor/Board/SE**yyyyy** /Config

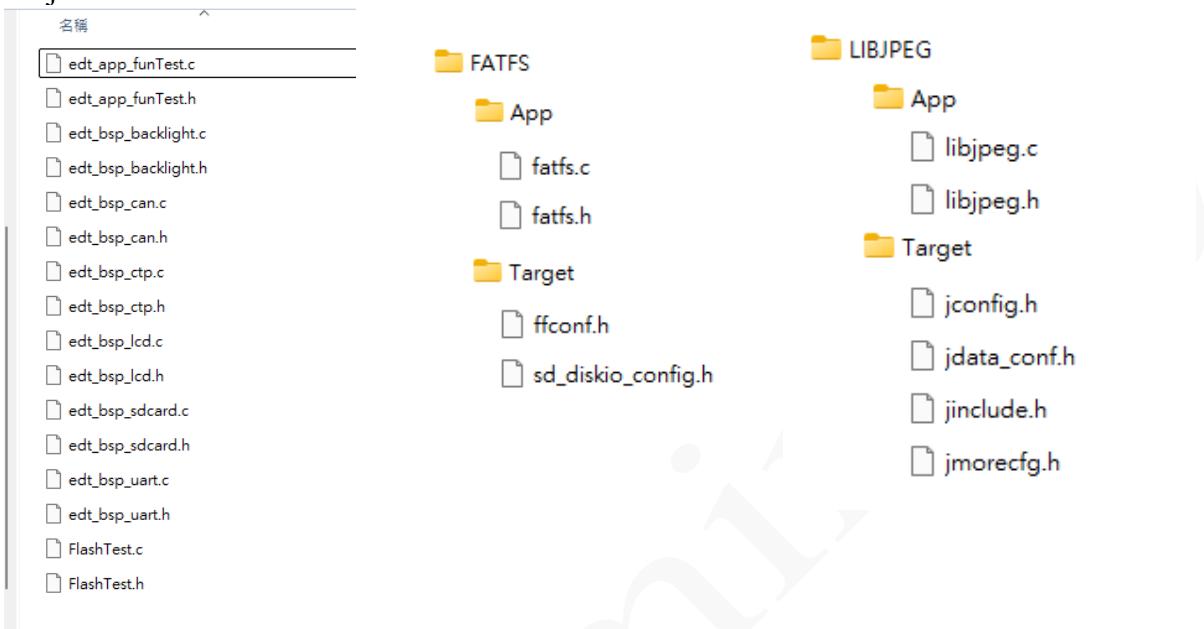



```
/* Define to prevent recursive inclusion -------------------------------------*/
#ifndef __SE21194_SYS_CONF_H
#define __SE21194_SYS_CONF_H

#ifdef __cplusplus
 extern "C" {
#endif

#include "main.h"

/****************** DISPLAY Private define************************/
#if (BoardPN == 43)
#define EVK043027B
#define  TFT_WIDTH            ((uint16_t)480)           /* LCD PIXEL WIDTH
#define  TFT_HEIGHT           ((uint16_t)272)           /* LCD PIXEL HEIGHT
#define PartNumber_STRING "ETEMB043027XDRAL"
#define MXT336U  // U599 series For 4.3 and 5.0
#define TS_ORIENTATION ((uint8_t) 0x04)     //TS_SWAP_Y
//#include "lcd/et043027_LCD.h"
#endif

#if (BoardPN == 50)
#define EVK050027B
#define  TFT_WIDTH            ((uint16_t)800)           /* LCD PIXEL WIDTH
#define  TFT_HEIGHT           ((uint16_t)480)           /* LCD PIXEL HEIGHT
#define PartNumber_STRING "ETEMB050027XDRAL"
//#include "lcd/et050027_LCD.h"
#define MXT336U  // U599 series For 4.3 and 5.0
#endif

#if (BoardPN == 57)
#define EVK057027B
#define  TFT_WIDTH            ((uint16_t)640)           /* LCD PIXEL WIDTH
#define  TFT_HEIGHT           ((uint16_t)480)           /* LCD PIXEL HEIGHT
#define PartNumber_STRING "ETEMB057027XDRAL"
#define MXT640U    // U599 series For 5.7 and 7.0
//#include "lcd/et057027_LCD.h"
#endif

#if (BoardPN == 70)
#define EVK070027B
#define  TFT_WIDTH            ((uint16_t)800)           /* LCD PIXEL WIDTH
#define  TFT_HEIGHT           ((uint16_t)480)           /* LCD PIXEL HEIGHT
#define PartNumber_STRING "ETEMB070027XDRAL"
#define MXT640U  // U599 series For 5.7 and 7.0
//#include "lcd/et070027_LCD.h"
#endif


#define USE_BL_PWM
//#define USE_LCD_SleepFunction
```

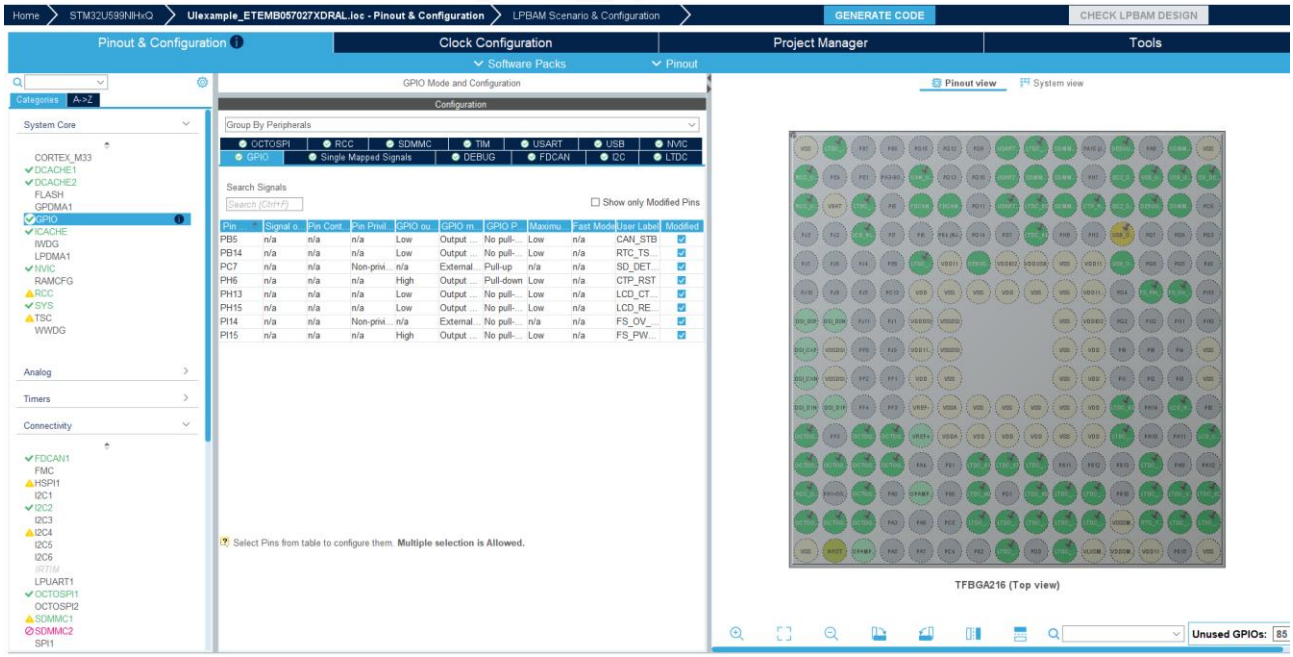
***yyyyy** is the PCB no.

## 4.3 Low level Drivers for the On-board Interfaces

You will find the low-level drivers for the On-Board Interfaces in the following directory:
Project xxx \Drivers\Vendor\Driver
Project xxx \ FATFS
Project xxx \ LIBJPEG

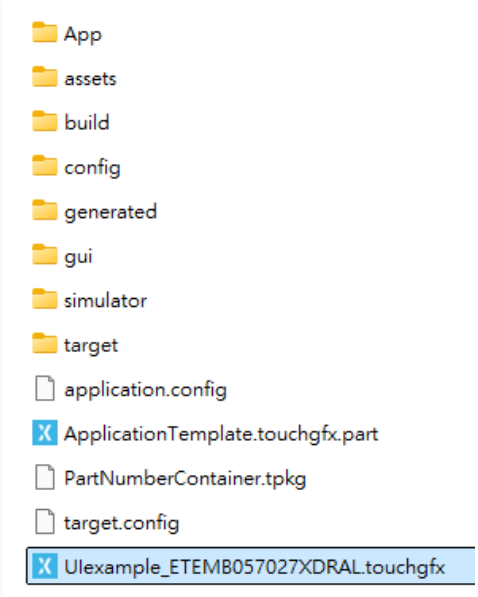



If you need to modify the parameter initialization used by a specific interface, please refer to the IoC file.



# 5. The TouchGFX Project

Navigate to the ‘Projectname’\TouchGFX Directory.



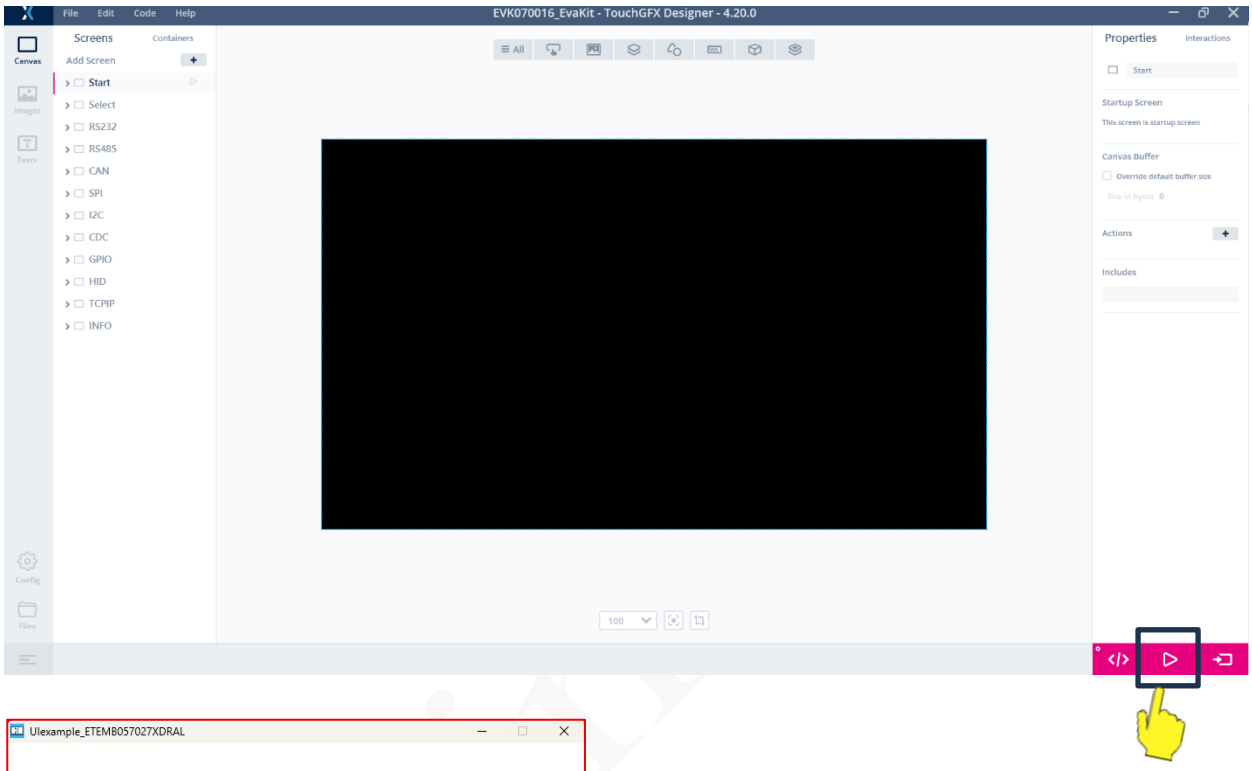

Double clicks on the TouchGFX project and navigate around, check the elements on the different Screens and the Interactions.

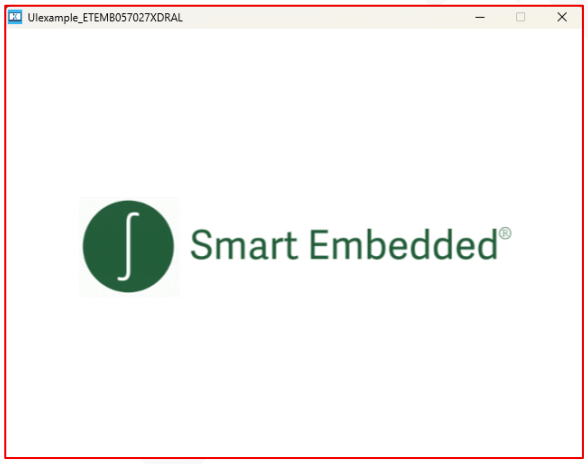
Preliminary

## 5.1 Main TouchGFX Designer Screen

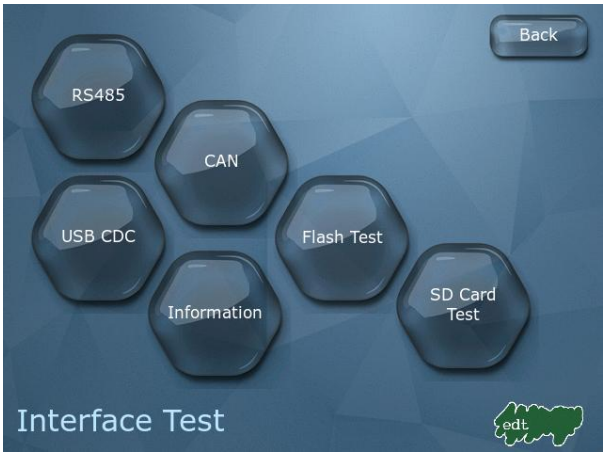
**Run Simulator**

Click on 'Run Simulator' and the Simulator will start after the Project has been compiled, and you can now use the mouse to navigate around in the application. The screen will automatically move to "Interface Test" screen after 1.5 seconds.




after 1.5 seconds



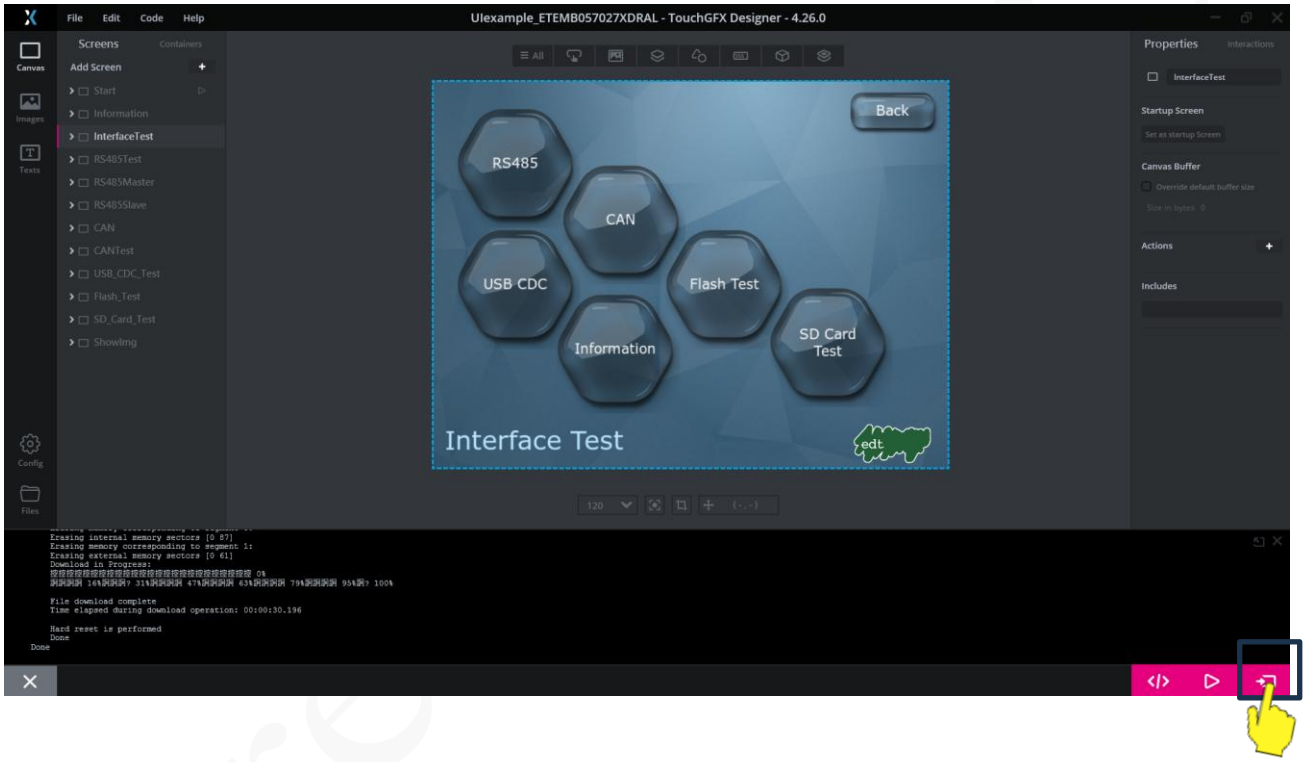
## 5.2 Run the Project on the Target Hardware

Now it is time to connect the ST-LINK Programming adapter to the PC using the USB cable and the 6-pin connector to the ST-LINK Connector on the PCB. Also connect Power Source to Power.

**Double check that the Power Cable is connected correctly before applying power!**

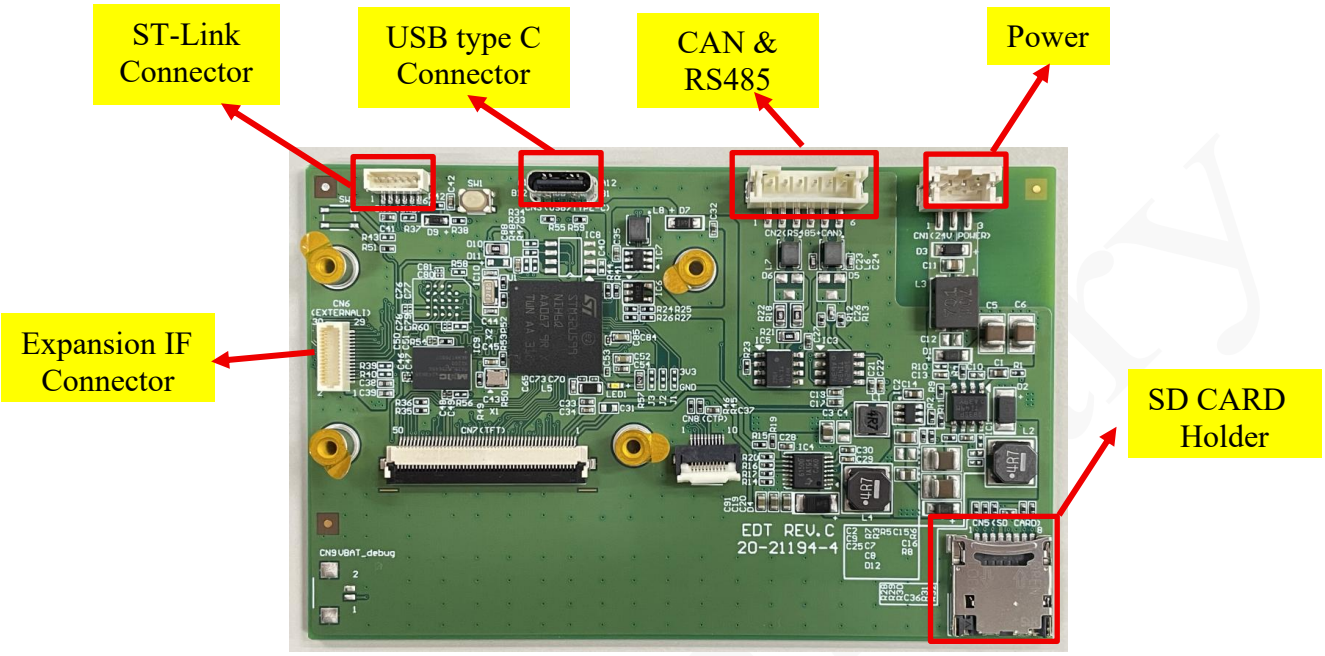
Click 'Run Target' and the Project will be compiled for the Target Processor and when compiled and linked the code will be downloaded to the Target.

Hint: Click on 'Detailed Log' in lower Right Corner to see the output from Compiler and Linker.



**Putting it all together**

Connect the Interface Cable you want to use.




## *5.3 Connection Diagram for RS485 and CAN Interface*

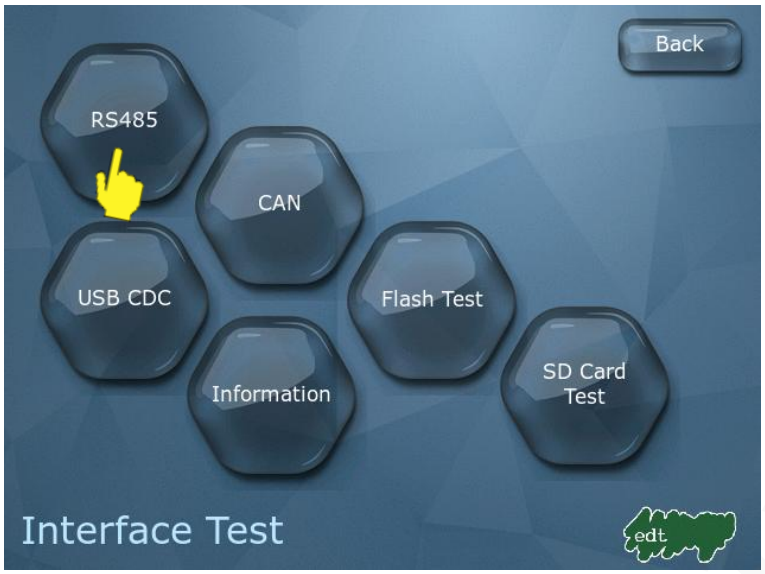
| CN2 (CAN & RS485) | Signal | RS485 to USB Converter DB9 | CAN to USB Converter DB9 |
|---|---|---|---|
| Pin 1 | VSS | Pin 5 | |
| Pin 2 | RS485-A | Pin 1 | |
| Pin 3 | RS485-B | Pin 2 | |
| Pin 4 | VSS | | Pin 3 |
| Pin 5 | CANL | | Pin 2 |
| Pin 6 | CANH | | Pin 7 |

Connect a USB-RS485 Serial Converter to the PC, start your favorite Terminal Program (we use Serial Port Utility and Putty) and select the Serial Parameters as follows:

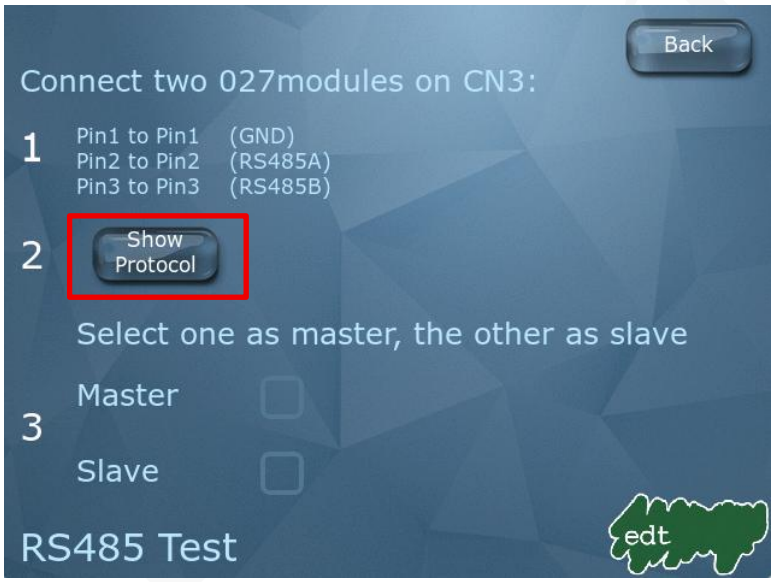
Baud rate: 115200
Parity: None
Data bits: 8
Stop bits: 1

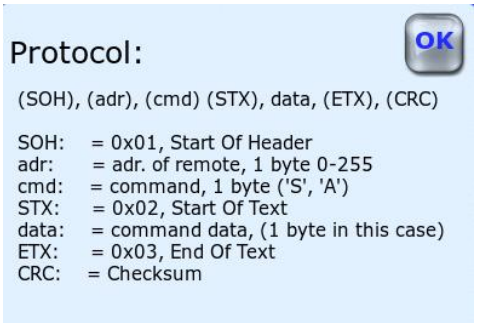
## 5.4 RS485 Screen

5.4.1 Press RS485 Button to enter the RS485 test screen.

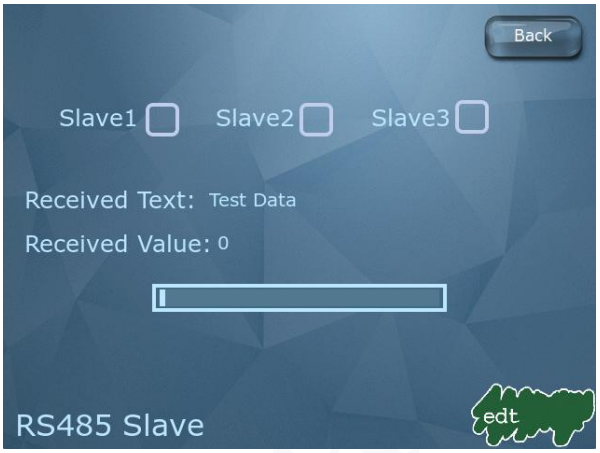



5.4.2. Press the “Show Protocol” button to see the RS485 protocol details.



5.4.2 **Select "Master" or "Slave" according to the device role.**
The system will display a different test screen based on the selected role.

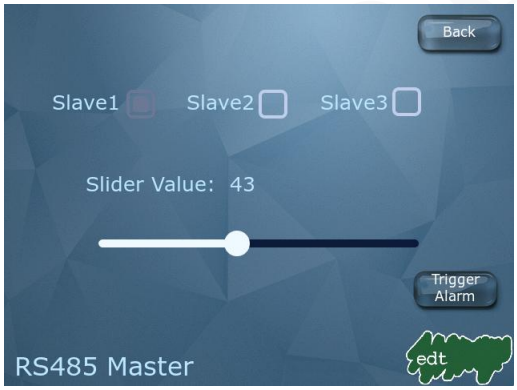



**RS485 Master Screen:**
When Master is selected, the RS485 Master screen is displayed.

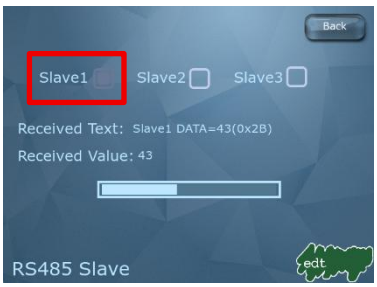
1. Select one or more slave devices (Slave1 / Slave2 / Slave3).
2. Adjust the slider to set the transmission value.

**RS485 Slave Screen**
When Slave is selected, the RS485 Slave screen is displayed.

1. Select the corresponding slave ID (Slave1 / Slave2 / Slave3).
2. The received data is shown in the Received Text and Received Value fields.
3. Verify that the received value matches the transmitted value from the master.



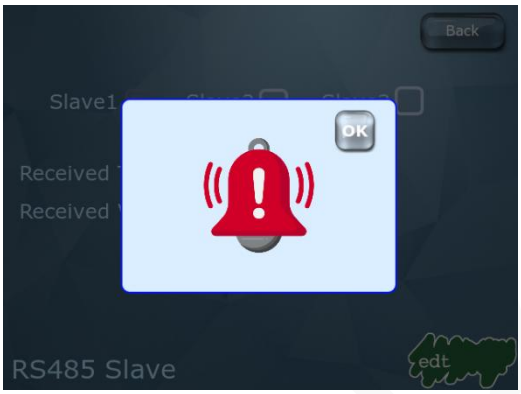
If an incorrect slave is selected, only the received text is showed, and the progress bar will not change.




When the correct slave is selected, the progress bar updates correctly and valid data is received.

In this demo, one master can communicate with up to three slave devices simultaneously for testing.

5.4.3 When the master presses “Trigger Alarm”, a warning dialog is displayed on the slave screen.

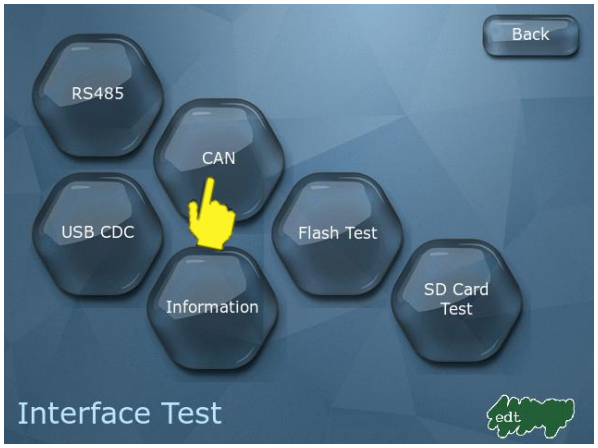


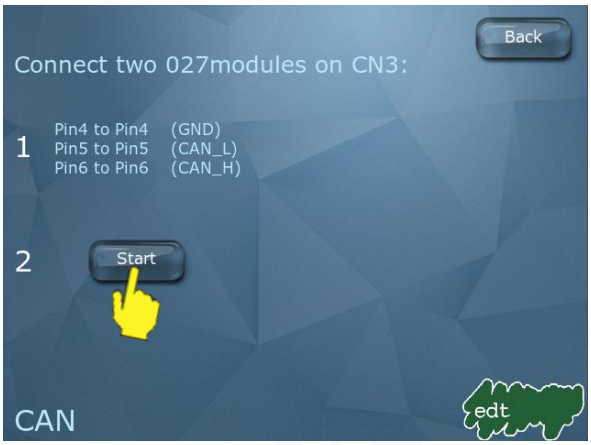

Note:
For more detailed implementation information, please refer to the program flowchart to locate the corresponding section.

Preliminary

## 5.5 CAN Screen

5.5.1 Press CAN Button to enter the CAN test screen

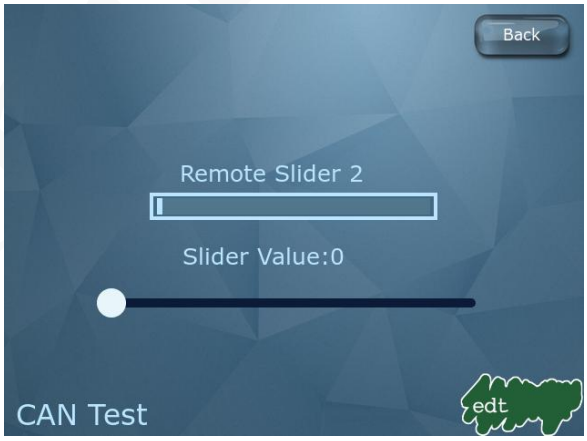



Baud rate: 125000bit/s
Frequence setting is 20MHz

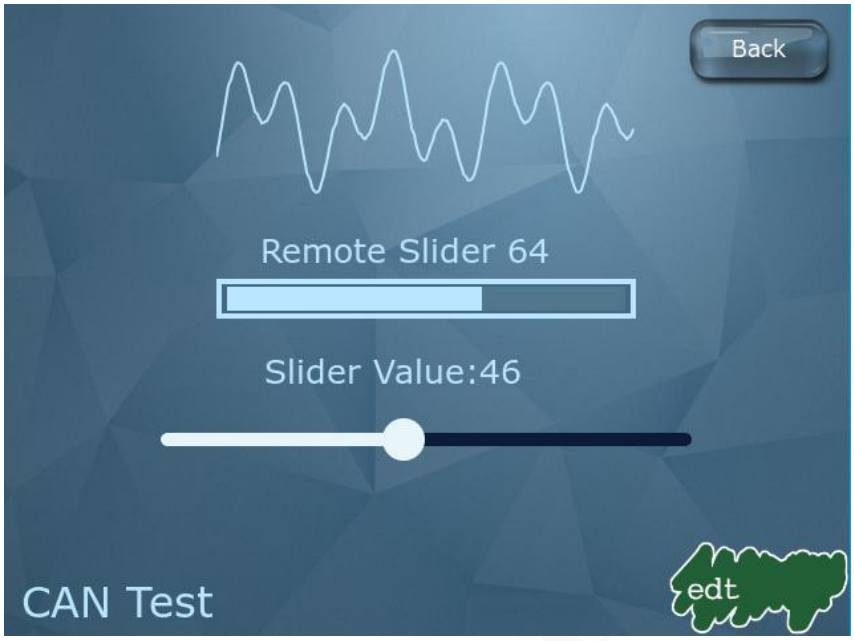
Please refer to ioc file for more details.

| Bit Timings Parameters | |
|---|---|
| Nominal Prescaler | 4 |
| Nominal Time Quantum | 200.0 ns |
| Nominal Time Seg1 | 31 |
| Nominal Time Seg2 | 8 |
| Nominal Time for one Bit | 8000 ns |
| Nominal Baud Rate | 125000 bit/s |




Preliminary

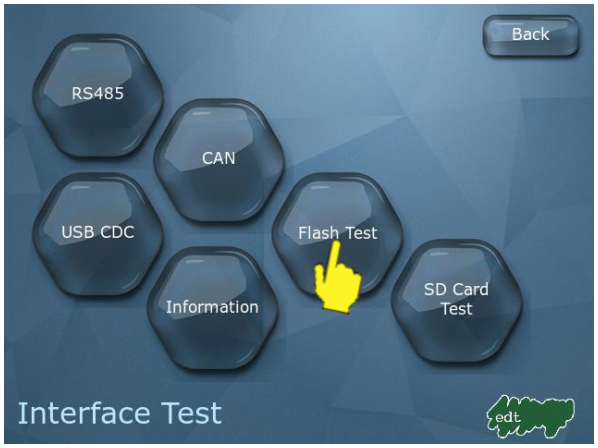
5.5.2 Slide the slider to send the value via CAN communication.
5.5.3 When a value is received via CAN communication, it is displayed on the Remote Slider and recorded in the chart above.

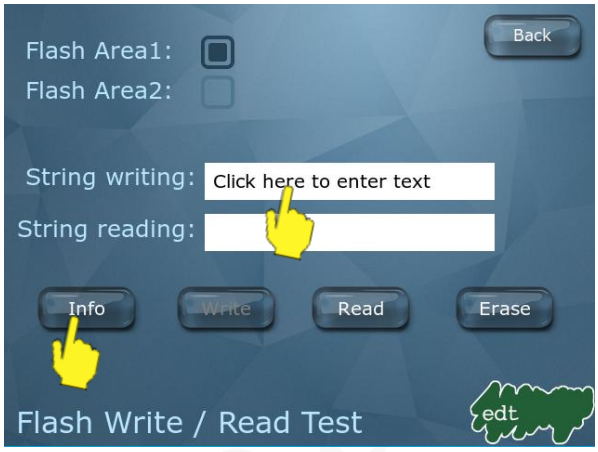



Note:
For more detailed implementation information, please refer to the program flowchart to locate the corresponding section.

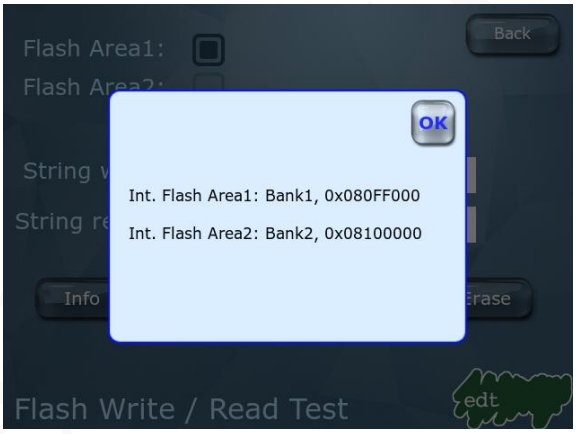
## 5.6 Flash Test Screen

The MCU contains two internal flash banks, each with a capacity of 1 MB.
This demo provides a simple example that allows customers to write string data directly into these two flash banks for testing.

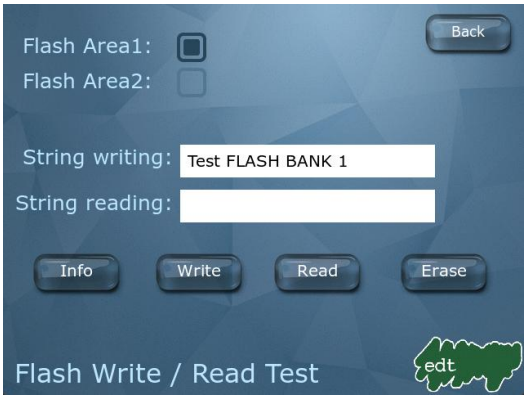



5.6.1 Press the "Info" button to identify which flash bank and location the data is written to.

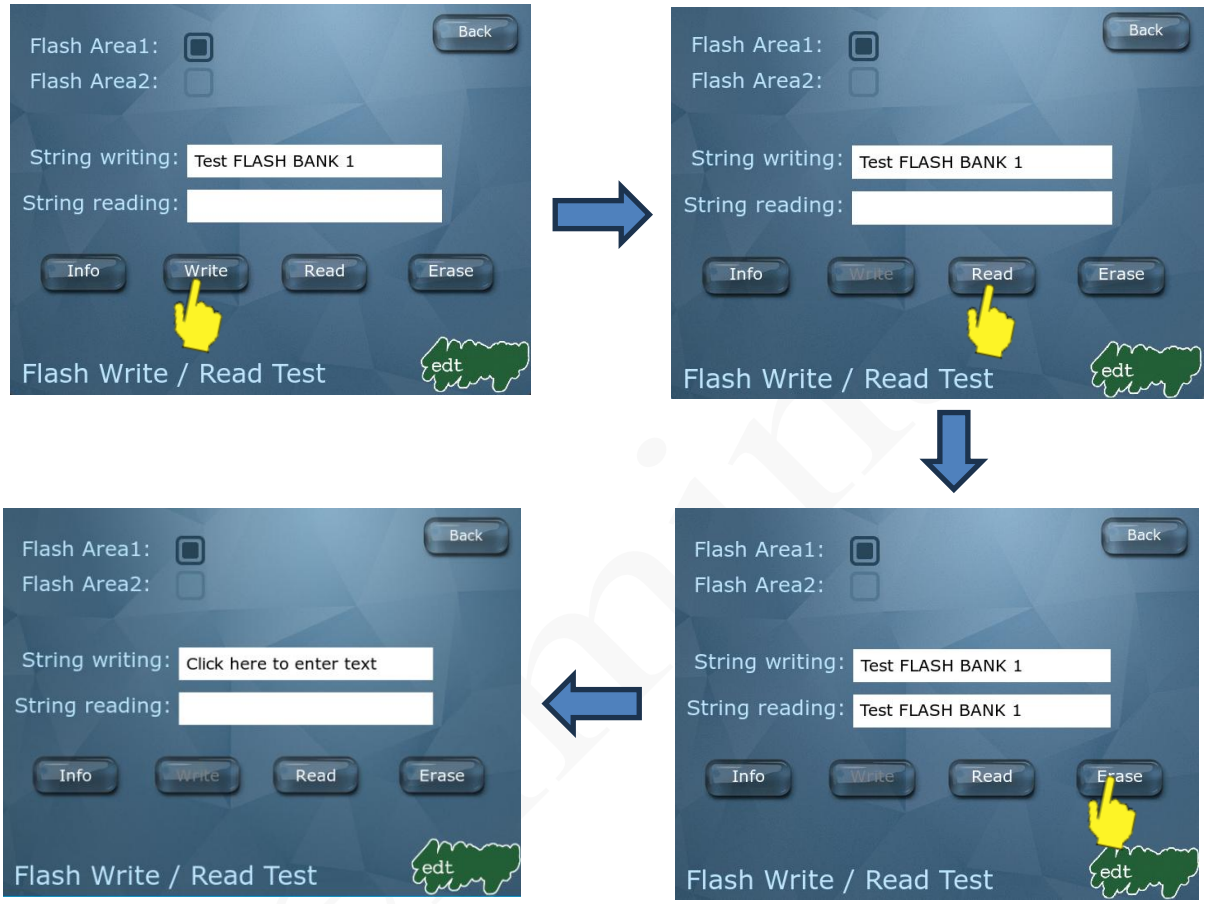



5.6.2 Press the input field beside "String writing" to open the keyboard.
Please enter text less than 24 characters and press "OK" to confirm.
The written string will be shown in the corresponding field.

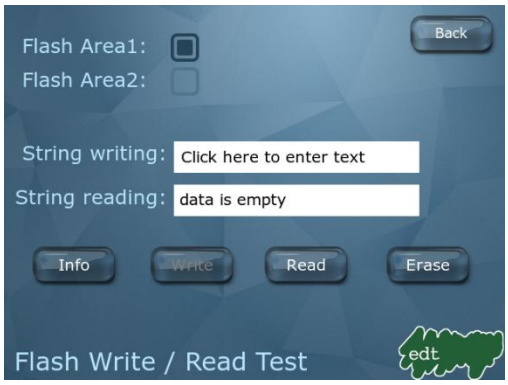


5.6.3 Press the "Write" button to store the entered string into internal Flash Bank 1. After pressing the "Read" button, the data stored in Flash Bank 1 will be displayed in the white text box next to "String reading".
Press the "Erase" button to delete the data stored in Flash Bank 1.

Select "Flash Area2". The operation procedure is the same as for Flash Area1.



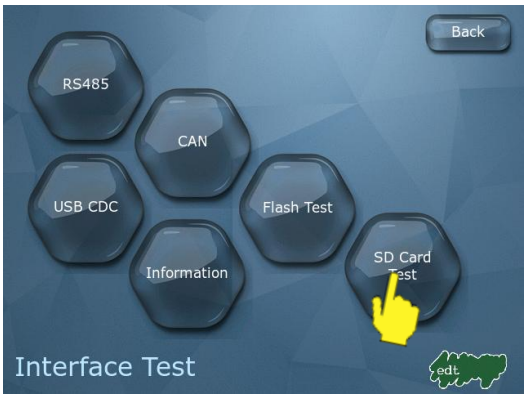

Note: If no text has been written and the "Read" button is pressed, the message "data is empty" will be displayed when the flash memory is blank.

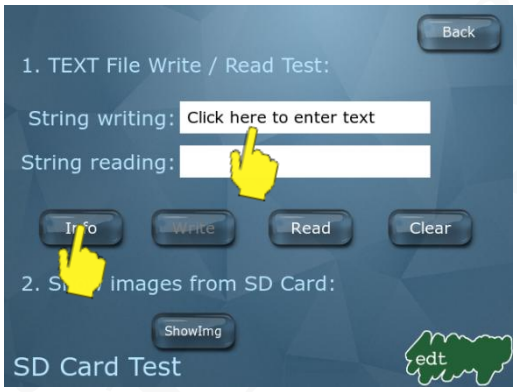



Note:
For more detailed implementation information, please refer to the program flowchart to locate the corresponding section.

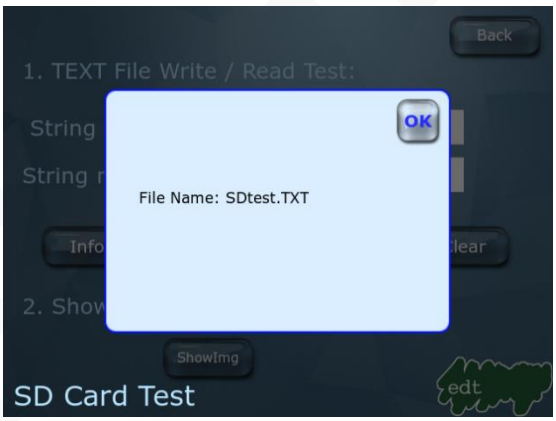
## 5.7 SD Card Test

The SD Card Test Demo provides an example of how to write and read text files on an SD card. In addition, to display images at a specified position, a dedicated window is created in the screen. The demo uses ST's official software decoding method to read image files (BMP and JPG) stored on the SD card.
To ensure proper display performance and system efficiency, it is recommended that the image resolution of BMP and JPG files does not exceed 480 × 272.

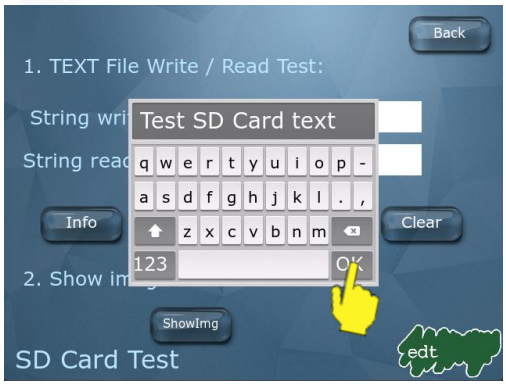


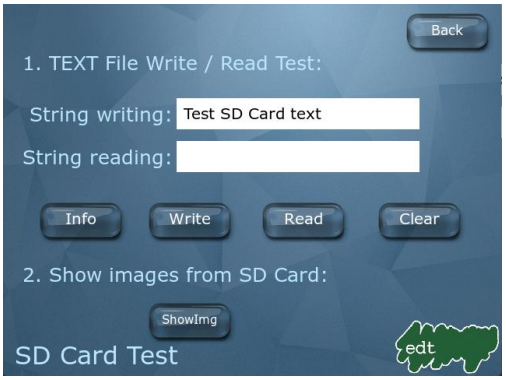




5.7.1 Press the "Info" button to view the file name and location where the data is stored on the SD Card.

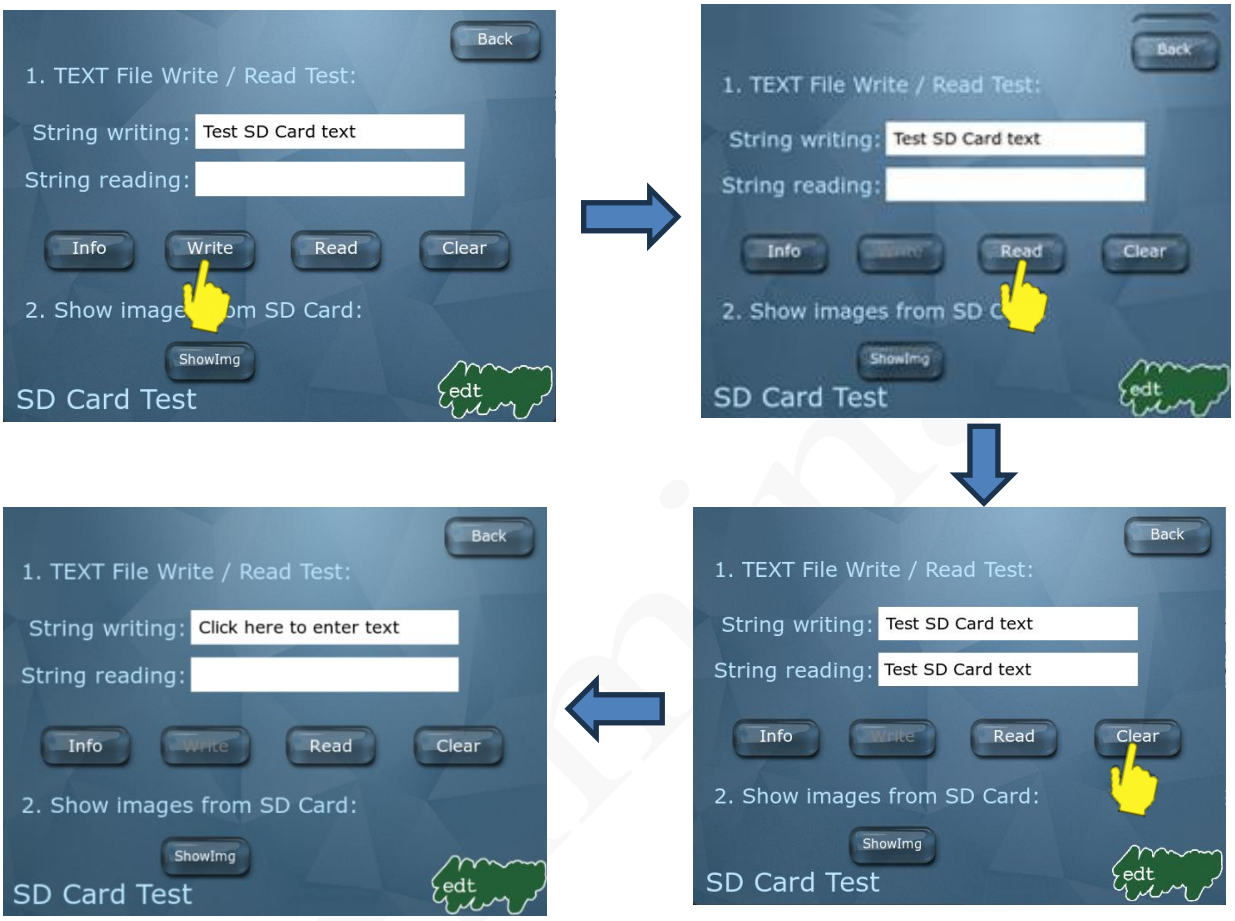



5.7.2 Press the input field beside "String writing" to open the keyboard.
Please enter text less than 24 characters and press "OK" to confirm.
The written string will be shown in the corresponding field.

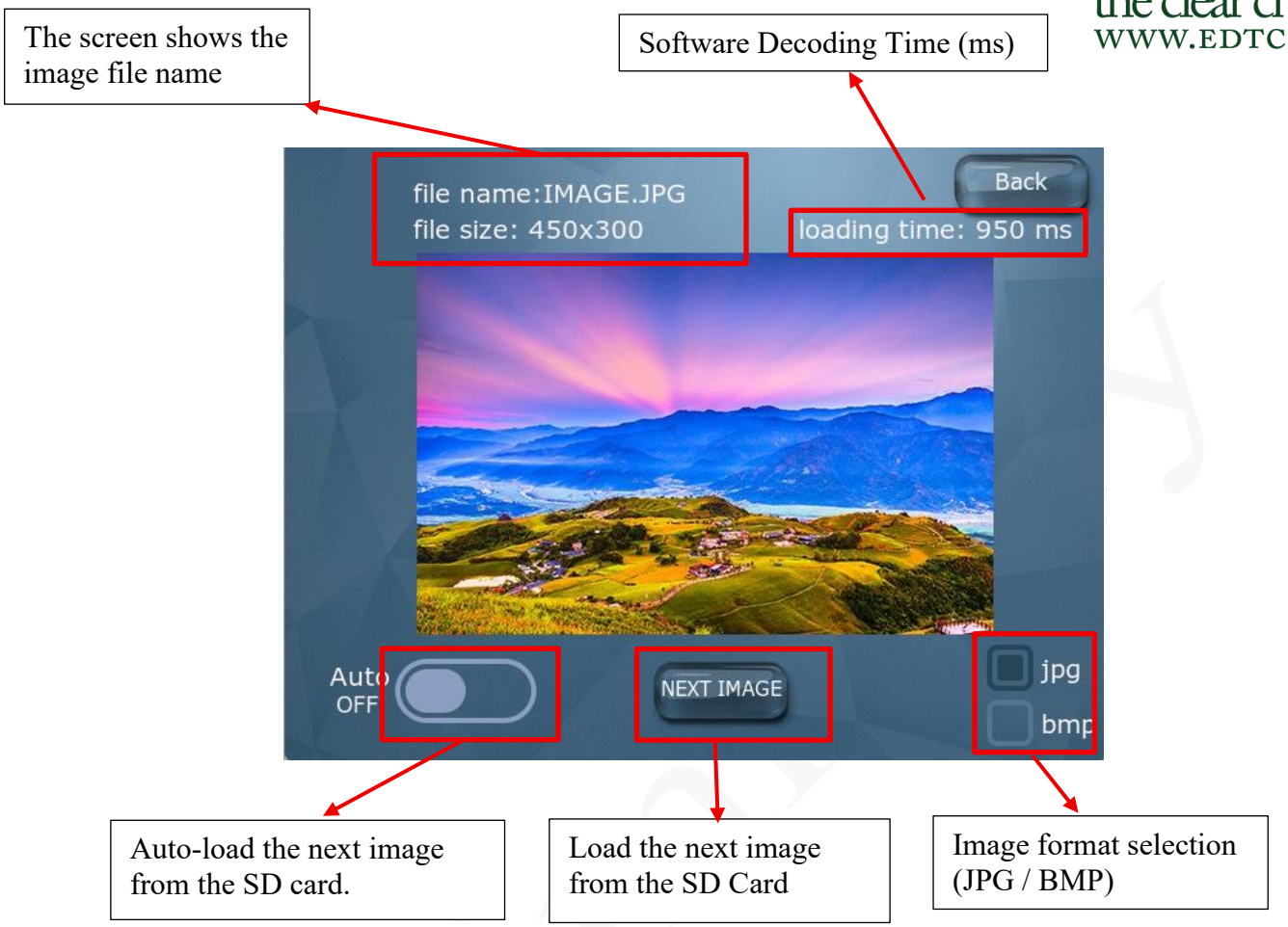


5.7.3 Press the "Write" button to store the entered string into SD Card.
After pressing the "Read" button, the data stored in SD Card will be displayed in the white text box next to "String reading".
Press the "Clear" button to clear only the text in the "String reading" field.




5.7.3 Press "ShowImg" to open the image decoding and display screen for BMP or JPG files.




Note:

To display decompressed BMP/JPG images at a specified position, the framebuffer and bitmap cache must be configured in **TouchGFXConfiguration.cpp**. In the touchgfx_init() function, the internal SRAM is divided into two regions:

- The first region is reserved for the framebuffer
- The remaining memory is used as a bitmap cache for decoded image data

A memory offset is applied after the framebuffer to allocate space for the bitmap cache:

```
// Use internal RAM within safe limits
static uint32_t frameBuf = (uint32_t)0x20170000; // Use internal RAM
uint32_t offset       = (2 * 640 * 480); /* Framebuffer Region size (RGB565 = 2 bytes per pixel) */
uint8_t *bmpCache     = reinterpret_cast<uint8_t*>(frameBuf) + offset;
```

The bitmap cache is then registered using:

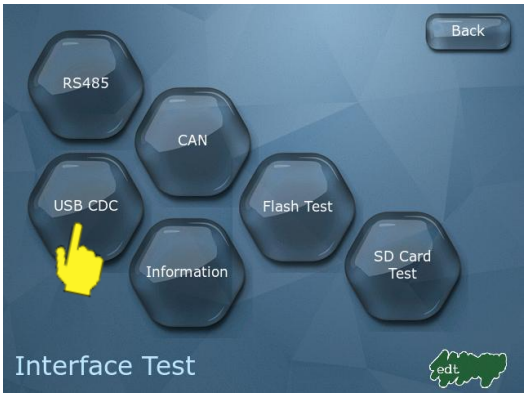
```
Bitmap::registerBitmapDatabase(BitmapDatabase::getInstance(), BitmapDatabase::getInstanceSize(),
                               (uint16_t*)(bmpCache), bitmapCacheSize, 4);
```

This configuration ensures that the decompressed image buffer can be stored in SRAM and correctly transferred into the framebuffer for display via DMA2D.

This code section will be reset and overwritten when the .ioc project regenerates the source files. Please refer to readme.txt and copy the required code back into TouchGFXConfiguration.cpp after regeneration. ( Please refer to Section 4 for the file location.)

## 5.8 USB CDC Screen

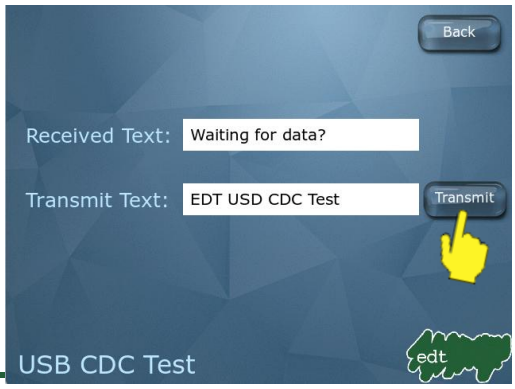
5.8.1 Press USB CDC Button to enter the USB CDC test screen

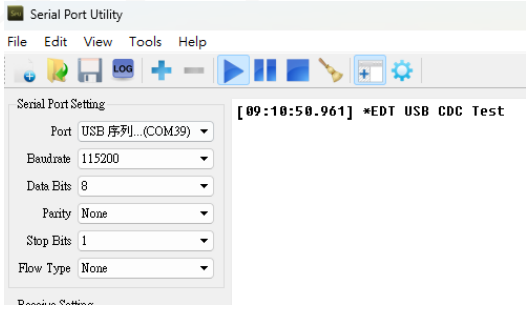


USB CDC serial parameters are as follows:

Baud rate: 115200 Parity: None Data bits: 8 Stop bits: 1

5.8.2 Press the input field beside "Transmit Text" to open the keyboard.
Please enter text less than 24 characters and press "OK" to confirm.
The written string will be shown in the corresponding field.

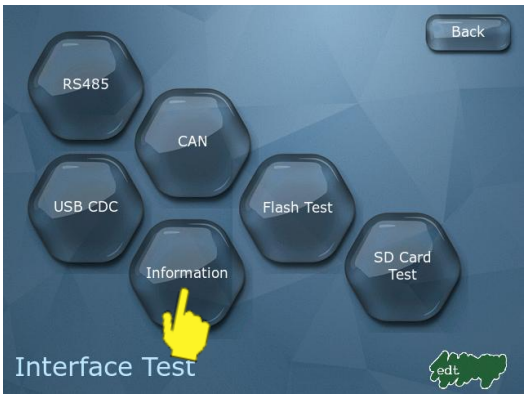



You may use a PC serial terminal tool (e.g., Tera Term or PuTTY) to send and receive data.

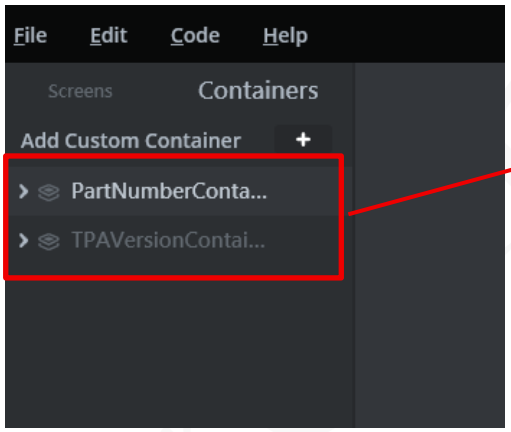
## 5.9 Information Screen

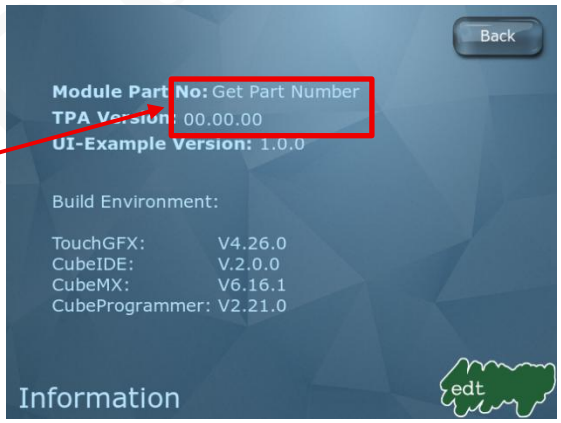
The Information screen provides the product model name, the TPA firmware version, and related software version information.



Note:

Two custom containers are added to this screen to conveniently retrieve and display information from se21194_sys_conf.h.



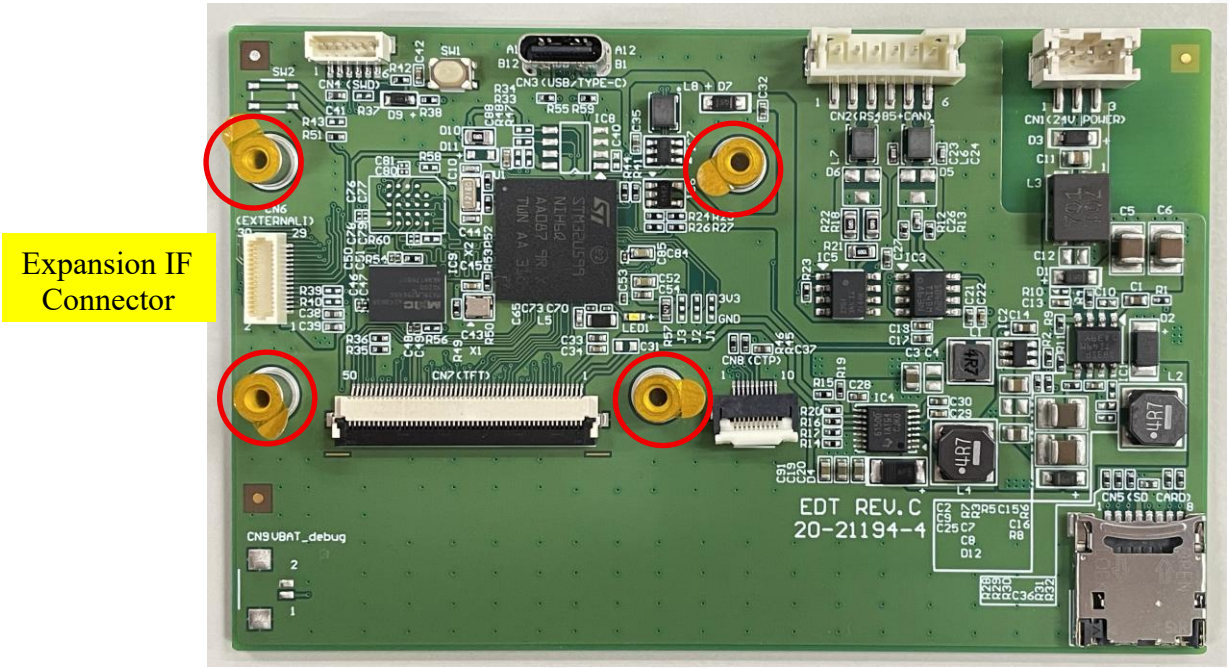

```
/** Board support package version: 1.0.3 (tpa file) ****************/
#define __EDT_AT_VERSION_MAIN  (0x01UL) /*!< [31:24] main version */
#define __EDT_AT_VERSION_SUB1  (0x00UL) /*!< [23:16] sub1 version */
#define __EDT_AT_VERSION_SUB2  (0x03UL) /*!< [15:8] ?sub2 version */
#define __EDT_AT_VERSION ((__EDT_AT_VERSION_MAIN << 24)\
                          |(__EDT_AT_VERSION_SUB1 << 16)\
                          |(__EDT_AT_VERSION_SUB2 << 8 ))

inline uint32_t EDT_GetTPAVersion(void) { return __EDT_AT_VERSION; }
```

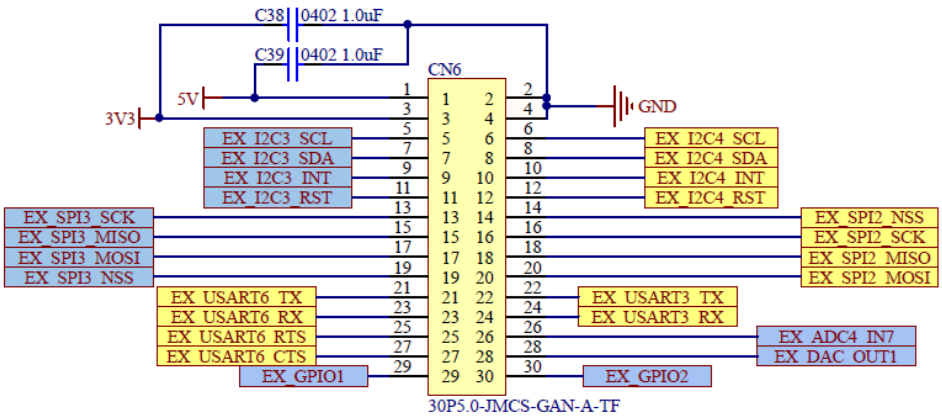
```
#if (BoardPN == 57)
#define EVK057027B
#define  TFT_WIDTH            ((uint16_t)640)          /* LCD PIXEL WIDTH          */
#define  TFT_HEIGHT           ((uint16_t)480)          /* LCD PIXEL HEIGHT         */
#define PartNumber_STRING "ETEMB057027XDRAL"
#define MXT640U    // U599 series For 5.7 and 7.0
//#include "lcd/et057027_LCD.h"
```

# 6. Expansion connector

The expansion connector provides an interface for connecting external modules and additional peripherals. It allows users to extend the functionality of the board by accessing communication signals and I/O resources.

The expansion board can also be secured with screws to ensure a stable connection on the product, making it suitable for production use as well.




See next page for Expansion connector pinning description and alternate functions.

Please refer to the figure below for the detailed pin definitions.

| PIN NO. | SYMBOL | Alternate Functions | | | | |
|---|---|---|---|---|---|---|
| | | MCU I/O Port | GPIO | ADC/DAC | UART | TIMER |
| 1 | +5V OUT | - | - | | | |
| 2 | GND | - | - | | | |
| 3 | +3.3V OUT | - | - | | | |
| 4 | GND | - | - | | | |
| 5 | EX_I2C3_SCL | PG_7 | GPIO | | LPUART1_TX | |
| 6 | EX_I2C4_SCL | PB_6 | GPIO | | USART1_TX | |
| 7 | EX_I2C3_SDA | PG_8 | GPIO | | LPUART1_RX | |
| 8 | EX_I2C4_SDA | PB_7 | GPIO | | USART1_RX | |
| 9 | EX_I2C3__INT | PG_6 | GPIO | | LPUART1_RTS/_DE | |
| 10 | EX_I2C4__INT | PE_4 | GPIO | | | TIM3_CH2 |
| 11 | EX_I2C3_RST | PG_5 | GPIO | | LPUART1_CTS | |
| 12 | EX_I2C4_RST | PE_1 | GPIO | | | TIM17_CH1 |
| 13 | EX_SPI3_SCK | PG_9 | GPIO | | USART1_TX | |
| 14 | EX_SPI2_NSS | PI_0 | GPIO | | | TIM5_CH4 |
| 15 | EX_SPI3_MISO | PG_10 | GPIO | | USART1_RX | |
| 16 | EX_SPI2_SCK | PI_1 | GPIO | | | |
| 17 | EX_SPI3_MOSI | PG_11 | GPIO | | USART1_CTS | TIM15_CH2 |
| 18 | EX_SPI2_MISO | PI_2 | GPIO | | | TIM8_CH4 |
| 19 | EX_SPI3_NSS | PG_12 | GPIO | | USART1_RTS | |
| 20 | EX_SPI2_MOSI | PI_3 | GPIO | | | |
| 21 | EX_USART6_TX | PJ_3 | GPIO | | | |
| 22 | EX_USART3_TX | PC_4 | GPIO | ADC1_IN13<br>ADC2_IN13<br>ADC4_IN22 | | |
| 23 | EX_USART6_RX | PJ_4 | GPIO | | | |
| 24 | EX_USART3_RX | PC_5 | GPIO | ADC1_IN14,<br>ADC2_IN14<br>TAMPIN4 | | |
| 25 | EX_USART6_RTS | PJ_5 | GPIO | | | |
| 26 | EX_ADC4_IN7 | PG_0 | GPIO | | | |
| 27 | EX_USART6_CTS | PJ_7 | GPIO | | | |
| 28 | EX_DAC1_OUT1 | PA_4 | GPIO | ADC1_IN9<br>ADC2_IN9<br>ADC4_IN9 | | |
| 29 | EX_GPIO1 | PJ_2 | GPIO | | | |
| 30 | EX_GPIO2 | PJ_6 | GPIO | | | |

In this product, two different expansion boards are available to support various application requirements:

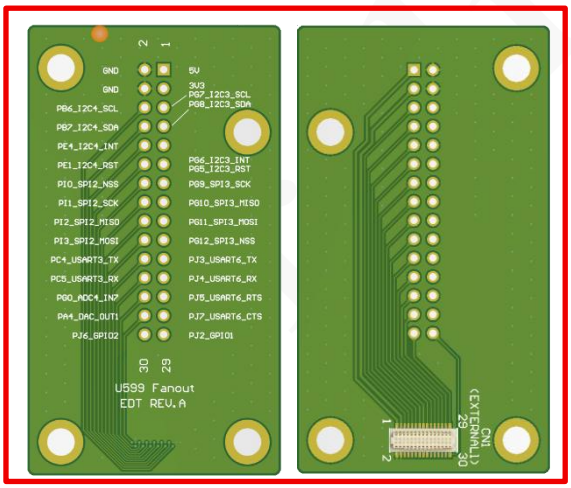
- **Fanout board**
  The Fanout Board is a passive expansion board designed to provide easy access to module pins. It allows users to conveniently connect signals through solder wiring or a standard **0.1-inch connector** for evaluation and prototyping.

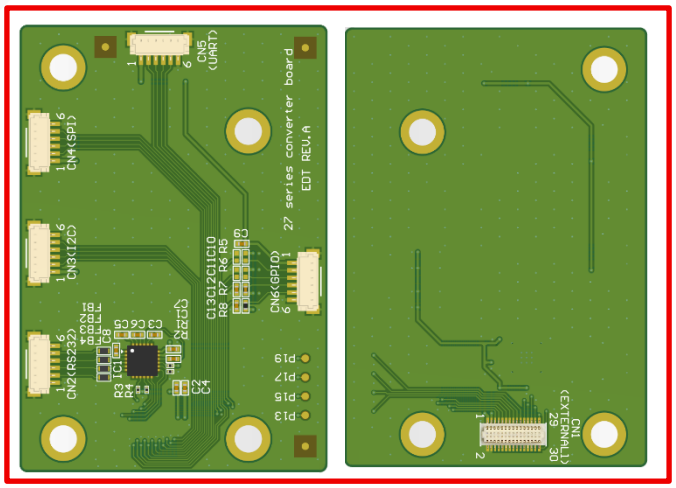
- **27 Series Connector Board**
  The 27 Series Connector Board includes **five 6-pin connectors** and provides multiple communication interfaces, including:
  - RS232 UART level signals
  - I2C interface
  - SPI interface
  - 3.3V UART
  - 4-bit GPIO expansion

This board is intended for connecting external peripherals and performing interface communication tests. Users can select the appropriate expansion board depending on the intended use case.

**Note:** Please ensure the correct expansion board is connected and follow the pin assignment specification before operation.

**Fanout board**




**27 Series Connector Board**